Software Engineering Lab Release — 1

I. PROBLEM STATEMENT

Desktop productivity applications (e.g., MS Excel, VS Code) ship with static User Interfaces (UIs) containing upwards of 500+ individual commands. However, standard usage telemetry indicates that an individual user typically interacts with **less than 10%** of these features daily. This "one-size-fits-all" approach forces users to navigate constantly cluttered workspaces.

This static UI architecture creates specific, measurable pain points for three stakeholders:

- **Novices / Junior Users:** Face high Cognitive Load. A beginner spends an estimated **10–15 minutes daily** visually scanning crowded ribbons just to locate basic tools, resulting in a steep learning curve and "feature blindness."
- **Power Users:** Suffer from increased Task Completion Time (TCT). Advanced users waste hundreds of redundant clicks per week physically switching between tabs (e.g., jumping between the 'Home' and 'Insert' tabs **50+ times a day**) to access their specific workflow tools.
- **System Hardware:** Experiences Random Access Memory (RAM) and CPU bloat. The system is forced to load 100% of the frontend UI DOM nodes and backend logic modules into memory, even though **90%** of it goes unused during a session.

Why Existing Solutions are Insufficient?

Current workarounds, such as Microsoft's Quick Access Toolbar (QAT) or customizable workspaces, require *manual* configuration. Industry data shows **fewer than 5%** of users actively customize their UIs due to the required time and intent. Furthermore, modern UI frameworks (like Fluent UI) only offer screen-size responsiveness, lacking true behavioural adaptation.

Impact of Leaving the Problem Unsolved:

Failing to solve this costs organizations thousands of hours in lost productivity annually. Human-Computer Interaction (HCI) research from our proposal validates that forcing users to adapt to static software drops engagement by up to **31%** and decreases task efficiency by **22%** compared to systems that dynamically adapt to the user.

II. PROPOSED SOLUTION

We propose **AdaptiveFlow**, a self-optimizing desktop application framework. While Release 1 uses a spreadsheet interface as our primary testbed, AdaptiveFlow is designed as an application-agnostic framework. It autonomously restructures both the frontend UI and backend execution paths based on runtime telemetry, requiring zero manual configuration from the user.

Core Features & Capabilities:

- **Analytics Engine (Weighted Session Scoring):** The system tracks interaction events (button ID, timestamp) across the user's last K sessions. It computes a ranked usage score using a recency-weighted algorithm ($Score_i = \sum w_j \times C_i^j$), ensuring the UI adapts to current workflows rather than stale historical data.

- **Dynamic UI Generation (Usage-Based Clustering):** Instead of static ribbons, the system automatically clusters the top **20%** of highest-scoring tools into a dynamically generated, category-aware " **My Workflow**" tab. Conversely, the bottom **20%** of consistently unused features are banished to an "Overflow" tab.

- **Decoupled Architecture (MAPE-K):** Adaptation logic (Monitor-Analyze-Plan-Execute) runs completely independently of the core business logic. UI layouts are 100% dictated by a local, lightweight JSON configuration file rather than hardcoded DOM structures.

Novelty: The "Adaptive-First" Developer Paradigm & Optimizations

Current adaptive UIs merely hide unused elements using CSS (`display: none`), which fails to save system resources. AdaptiveFlow introduces a new developer standard:

- **Feature Toggle Flags:** Developers author functional modules wrapped in custom configuration flags (e.g., `ENABLE_PIVOT_TABLES = false`).

- **Runtime Debloating:** When the adaptation engine determines a feature group is unused, it toggles these flags off in the central config file. The system physically prevents the backend logic (e.g., heavy charting libraries or complex calculation modules) from loading into Random Access Memory (RAM).

- **Custom Installation States:** In the future, this config file can act as a deployment manifest. A user could perform a "Custom Installation" driven by their adaptive profile, installing only the **10–15%** of physical disk files they actually use, drastically reducing the application's disk footprint.

Expected Impact on Target Users:

- **For Novices:** Reduces cognitive load by decreasing rendered UI DOM nodes by an estimated **40% to 60%**, surfacing only essential tools.

- **For Power Users:** Cuts redundant navigation clicks by up to **50%** by clustering cross-tab workflows into a single customized ribbon.

- **For System Hardware:** Measurably reduces runtime memory consumption and initial application load times by preventing the initialization of unused background modules.

III. SYSTEM ARCHITECTURE

Our team is evaluating three distinct architectural prototypes to achieve self-adaptation. All three operate entirely locally (offline) to guarantee zero-latency performance and absolute data privacy.

Prototype A: The MAPE-K Engine (AdaptiveFlow)

This architecture completely decouples the UI representation from the adaptation logic using a formal feedback loop, ensuring the business logic remains untouched.

```
+------------------+  IPC Bridge  +------------------+
|                  | <-- log-click ---                |
| React Frontend   |              | Node.js Backend  |
| (Monitor/Execute)| -- ribbon-adapted -> | (Analyze/Plan)   |
|                  |              |                  |
+------------------+              +--------+---------+
                                           | JSON R/W
                                           v
                                  +------------------+
                                  | Local Disk       |
                                  | (Knowledge Base) |
                                  +------------------+
```

Prototype A: MAPE-K Architecture Diagram

- **React Context (Monitor/Execute):** Wraps UI elements to log user interactions. Listens for configuration updates to dynamically re-render the Document Object Model (DOM).
- **Electron IPC Bridge:** The secure communication layer passing asynchronous messages between the frontend UI and the operating system.
- **MAPE-K Backend (Analyze/Plan):** Evaluates click thresholds (e.g., > 3 for Favorites, > 10 total app clicks for Overflow). Modifies the JSON tree state.
- **Telemetry Storage (Knowledge):** `adaptive_telemetry.json` persists user behavior and pinned preferences across application restarts.

Prototype B: Usage-Based Clustering

This architecture focuses on grouping frequently used features by their inherent logical categories to maintain workflow familiarity.

```
+-------------+  Click Event  +-------------+
| User UI     | ------------> | Command     |
|(Interactions)|              | Usage       |
+------+------+              | Tracker     |
       ^                     +------+------+
       |                            | Save/Load
       | Generated                  v
+------+------+              +-------------+
| Dynamic     | <-- Filter & | Adaptability|
| Ribbon      |  Categorize  | Engine      |
| Generator   |              +-------------+
+-------------+
```

Prototype B: Usage-Based Clustering Diagram

- **CommandUsageTracker:** Aggregates command execution counts locally over recent user sessions.
- **AdaptabilityEngine:** Filters commands passing a strict `MIN_USES` threshold and clusters them using predefined `CATEGORY_PREFIXES` (e.g., matching "Font_Bold" and

"Font_Italic").

- **DynamicRibbonGenerator:** Synthesizes the categorized data into a new, consolidated " **My Workflow**" tab, reducing cross-tab navigation.

Prototype C: Weighted Session Scoring

This architecture utilizes a mathematical decay model to ensure the UI adapts to *current* tasks, rather than outdated historical behavior.

```
+-------------+  Log Event    +-------------+
| UI Layer    | ------------> | Session     |
|(Interactions)|              | Manager     |
+------+------+              +------+------+
       ^                            | Last K Sessions
       |                            v
       | Reorder UI          +-------------+
+------+------+              | Analytics   |
| UI Transform| <-- Ranked    | Engine      |
| Engine      |   Scores      +-------------+
+-------------+
```

Prototype C: Weighted Session Scoring Diagram

- **Session Manager:** Groups interactions into defined time/activity blocks, maintaining only the last K sessions in storage.
- **Analytics Engine:** Computes a ranked score ($Score_i = \sum w_j \times C_i^j$) where recent sessions (w_j) are mathematically weighted heavier than older ones.
- **UI Transformation Engine:** Sorts the ranked array, isolating the Top 20% into a "Frequently Used" tab and purging the Bottom 20% to a "Least Used" tab.

IV. TECH STACK

To ensure absolute data privacy, zero-latency execution, and a native desktop feel, our team selected a modern, offline-first web technology stack wrapped in a native desktop container.

A. Frontend (UI & State): React.js (TypeScript) + Vite

Justification: React's component-based architecture and Context API are perfectly suited for our engine. It allows us to dynamically re-parent, reorder, and unmount UI elements (DOM manipulation) instantly via JSON state updates without reloading the window. Vite ensures fast development builds.

B. Backend (Adaptation Logic): Node.js (via Electron IPC)

Justification: Node.js handles the heavy lifting of the MAPE-K loops, array sorting algorithms, and session scoring math outside of the main browser thread, ensuring the UI remains perfectly responsive during adaptation cycles.

C. Desktop Environment: Electron.js

Justification: Essential for packaging our web application as a native, offline desktop executable. It provides the crucial bridge (IPC) allowing our React frontend to securely communicate with the OS-level local file system.

D. Database / Storage: Local JSON (Node fs / localStorage)
Justification: We deliberately avoided cloud databases (like Firebase or PostgreSQL) to guarantee 100% data privacy and offline capability. Telemetry (`adaptive_telemetry.json`) and configuration states are read/written directly to the user's local disk.

E. Collaboration & Version Control: GitHub & Discord
Justification: GitHub provides verifiably tracked code contributions and branch management, while Discord serves as our primary asynchronous communication channel.

V. TEAM ROLES

Our team has divided the workload to cover architectural engineering, core application development, and academic research.

Pranav Tej (CS24B057): Project Lead & Core Adaptation Architect

- *Responsibilities:* Designing the overarching MAPE-K engine, cross-tab teleportation logic, and the secure IPC bridge.
- *Release 1 Tasks Completed:* Engineered the `runAdaptationEngine` algorithm in `main.ts`, implemented local file system persistence for telemetry, and resolved cross-tab duplicate-button logic bugs.

Ankit Sinha (CS24B056): Research Lead & Documentation Specialist

- *Responsibilities:* Literature review, academic paper identification, and structuring project documentation to align with software engineering standards.
- *Release 1 Tasks Completed:* Spearheaded the project's research phase by identifying, analyzing, and synthesizing key academic papers on self-adaptive systems (e.g., the Rainbow Framework) and software debloating. Authored the foundational project proposal and managed the compilation of the Release 1 report.

Karthik (CS24B046): Backend Adaptation Researcher & UI Developer

- *Responsibilities:* Investigating backend optimization techniques and assisting with frontend layout scaffolding.
- *Release 1 Tasks Completed:* Conducted R&D on runtime backend modification and feature-flag toggling (the core target for Release 2's RAM-saving goals) to ensure feasibility. Assisted in scaffolding the initial React CSS layouts and component structure for the static Ribbon toolbar.

Bhargav (CS24B028): Core Functional Developer

- *Responsibilities:* Development of the underlying spreadsheet grid, formulas, and data structures.
- *Release 1 Tasks Completed:* Built the core MS Excel Clone interface (the testbed application) that the adaptive engine runs on top of, and successfully prototyped the secondary "Usage-Based Clustering" logic architecture.

Nilanjan Singh (CS24B029): Electron & Integration Specialist

- *Responsibilities:* Desktop wrapper setup, context bridging, and session-based analytics.
- *Release 1 Tasks Completed:* Initialized the Electron `main.ts` environment, configured

the `preload.ts` scripts for secure IPC communication between React and Node, and prototyped the mathematical "Weighted Session Scoring" architecture.

VI. AI USAGE DECLARATION

Ankit:

Prototyping a feature-flag based adapting Java desktop application — Code 100% AI.

Used: Antigravity (Claude Opus 4.6)

Prompt (an earlier version; actual conversation lost):

Role: You are a Senior Software Architect specializing in Self-Adaptive Systems and Modular Desktop Applications.

Task: Build a functional desktop application called "OmniTool" using Java with JavaFX. The app must be designed for resource-constrained environments, utilizing a Feature Toggle (Flipper) architecture instead of heavy runtime compilation.

Core Functionality Requirements:

Three Functional Pillars:

- Document Editor: Basic Markdown support and PDF export.
- Image Processor: Filters (Grayscale, Invert) and resizing logic.
- System Utils: A file renaming utility and a disk usage scanner.

Architecture Guidelines (Strict):

- **Module Decoupling:** Use the Command Pattern. UI Buttons must not contain logic; they must trigger a Command object.
- **Feature Flagging:** Every non-essential feature must be wrapped in a conditional check: `if (Flipper.isEnabled("FEATURE_ID"))`.
- **Lazy Loading:** Heavy libraries (like the PDF exporter or Image filters) must only be initialized/loaded if the Flipper returns true.

Required Modules to Implement:

- **Statistics Module:** Initialize a local SQLite database (`stats.db`). Create a background listener that increments a `usage_count` for a specific `FeatureID` every time its Command is triggered.
- **Flipper Module:** Read a local `config.json` file at startup containing a Map of `FeatureID (String)` to `isEnabled (Boolean)`. Provide a global `isActive(String id)` method that the UI and Logic layers can query.
- **The "Hidden Features" Tab:** Implement a dedicated UI view that lists every Feature ID currently marked as false in the Flipper. Provide a "Restore" button next to each to manually override the flag.
- **Weekly Adaptation Logic:** Create a simulated "Weekly Review" function. When triggered, it queries SQLite. Any feature with `usage_count < 3` is marked as `isEnabled: false` in the `config.json`.

Deliverables:

- Provide the source code for the Flipper, StatsTracker, and BaseCommand classes.

- Provide the UI layout code (FXML or Java) showing how a toolbar dynamically adds/removes buttons based on the `config.json`.
- Include a sample `config.json` with all feature flags set to true by default.

Nilanjan:

Code — 96.9% AI

Implementation of an adaptation algorithm on the MS Excel clone that utilizes a mathematical decay model to ensure the UI adapts to *current* tasks, rather than outdated historical behavior.

Prototype for Electron-based application for reordering of the elements based on usage.

- **Initial Request:** You asked for a simple Electron.js desktop app that moves a button based on user input.
  - **Output:** A 3-file setup (`package.json`, `main.js`, `index.html`) using absolute CSS positioning to move a button via coordinate inputs was provided.
- **Second Request:** You wanted to read those coordinates from a local text file instead of manual input.
  - **Output:** The code was updated to use the Node.js `fs` module to read `coords.txt` and move the button based on the values stored inside.
- **Third Request:** You scaled the project to 100 buttons (labeled 1–100), sized to fit 2–3 per row, with a requirement to track the click counts of each button in an external file.
  - **Output:** Storage was transitioned to a `data.json` file and a JavaScript loop was used to dynamically generate the buttons and update the JSON counts in real-time.
- **Fourth Request:** You asked to arrange these buttons in decreasing order based on their click counts.
  - **Output:** An `Object.entries().sort()` method was implemented that re-rendered the buttons in a "leaderboard" style every time a click occurred.
- **Fifth Request:** You refined the sorting logic to trigger only when the program restarts, rather than moving during the active session.
  - **Output:** The script was modified to perform the sort logic once during the initialization phase, keeping button positions static during use but updating the layout upon the next launch.
- **Final Request:** You requested a professional summary of the prototype's functionality for a weekly status report.
  - **Output:** A technical paragraph highlighting data persistence, startup sorting logic, and the dynamic UI generation was provided.

Pranav:
Sorted reordering of frequently used items and hiding mechanism for the rest in MS Excel to reduce RAM usage while app runs.

AI use: 95%

- **Prompt Entry 1**
  - **Tool:** Gemini
  - **Prompt:** "I'll keep giving you more, keep waiting and analyzing one by one. If you want more, you can just ask." *(Included contents of App.tsx, main.ts, preload.ts, and types.ts)*
  - **Purpose:** To supply the initial codebase context for an Electron and React application so the AI could analyze the foundation before making modifications.
  - **Outcome:** Useful.
- **Prompt Entry 2**
  - **Tool:** Gemini
  - **Prompt:** "Please check and let me know you need anything more? I'm ready. Be as cautious as possible" *(Included contents of Ribbon.tsx and SpreadsheetContext.tsx)*
  - **Purpose:** To provide the UI layout and state management files so the AI could identify exactly where to inject the telemetry and adaptation logic.
  - **Outcome:** Useful.
- **Prompt Entry 3**
  - **Tool:** Gemini
  - **Prompt:** "1. Adaptation should trigger live on the UI... 2. Actually I am not sure, but you can put it in a visible place... Also keep in mind, UI should look nice."
  - **Purpose:** To provide the TypeScript definitions file and answer the AI's design questions regarding adaptation triggers and button placement.
  - **Outcome:** Partially used.
- **Prompt Entry 4**
  - **Tool:** Gemini
  - **Prompt:** "As you can see, the name of the Group is going left... Also once a button is 'adapted', if a person uses it more frequently, then it should go more left..."
  - **Purpose:** To provide feedback on a UI bug and propose continuous left-to-right sorting.

  - **Outcome:** Useful.

- **Prompt Entry 5**
  - **Tool:** Gemini
  - **Prompt:** "1. The UI Bug: Group Labels Misaligned... Algo is fine for icons, but I hope you will respect their groups as well and don't misplace them"
  - **Purpose:** To correct the AI's assumption about when the CSS bug occurs and strictly enforce group boundaries.
  - **Outcome:** Useful.

- **Prompt Entry 6**
  - **Tool:** Gemini
  - **Prompt:** "No this don't work, Reset UI does nothing when clicked... This is absolutely horrible..."
  - **Purpose:** To report severe bugs where all buttons vanished and the local storage became corrupted.
  - **Outcome:** Useful.

- **Prompt Entry 7**
  - **Tool:** Gemini
  - **Prompt:** "Yes I think this works. But I found one interesting thing... I am attaching two screenshots that show what happens when I click Reset UI..."
  - **Purpose:** To report an oscillating layout bug that occurs when repeatedly clicking the Reset UI button.
  - **Outcome:** Partially used.

- **Prompt Entry 8**
  - **Tool:** Gemini
  - **Prompt:** "1 and 2 are perfect. 3 is also bit fine... I don't see any way in which I can selectively restore the groups... And cross tab adaptation"
  - **Purpose:** To request a selective restore feature and the implementation of cross-tab adaptation.
  - **Outcome:** Useful.

- **Prompt Entry 9**
  - **Tool:** Gemini
  - **Prompt:** "Now after adaptation there are duplicates. Also in more features when I go and press on any button its still not asking me to restore..."

  - **Purpose:** To complain about duplicate buttons and request explicit, visible restore buttons.
  - **Outcome:** Useful.

- **Prompt Entry 10**
  - **Tool:** Gemini
  - **Prompt:** [Terminal output showing a Vite esbuild error: The symbol "saveTelemetry" has already been declared]
  - **Purpose:** To resolve a syntax error caused by overlapping copy-paste operations.
  - **Outcome:** Useful.

- **Prompt Entry 11**
  - **Tool:** Gemini
  - **Prompt:** "Well you fixed the duplicates thing. But the feature where I will go to More Features and selectively restore some blocks is not there yet"
  - **Purpose:** To inform the AI that the frontend UI for the explicit restore feature was missing.
  - **Outcome:** Useful.

Karthik:
Established and validated the core UI adaptation methodology by implementing a dynamic "Hidden" section for less-frequently used elements, directly contributing to the final system's ability to reduce clutter and improve user-centric interface organization.

AI use: 100%

- **Tool:** Cursor
  Prompt: "Create a prototype of the application such that it should be able to hide the icons in the nav bar at a particular location and use only node related tech stack and there should be a designated place to hide the icons under that when the user is not using that icon not so much"
  Purpose: To scaffold an initial web prototype utilizing Node-related technologies, featuring a navigation bar capable of hiding less-used icons in a specific designated area.
  Outcome: Useful. The AI successfully generated a React and TypeScript prototype using Vite. It implemented the requested UI, including a collapsible "Hidden" section and a "Customize" drawer for managing icon visibility.

- **Tool:** Cursor
  Prompt: "Ok fine also remember one thing not to create any of the folders in the main root just navigate to the folder named cursor_projects and then do it using the mkdir or anything equivalent"
  Purpose: To enforce local directory management constraints, ensuring the generated application files were placed within a specific `cursor_projects` directory rather than the root folder.

Outcome: Useful. Although the AI initially encountered an error due to the shell session being inside the project folder, it successfully navigated back, stopped leftover processes, and moved the prototype into the requested `cursor_projects` directory while updating the README paths.

- **Tool:** Cursor
  Prompt: "Now give all these work in the textual format to add it in the weekly status report for the submission"
  Purpose: To synthesize the development work completed so far into a formatted summary suitable for a weekly academic or project status report.
  Outcome: Useful. The AI generated a comprehensive report detailing the design, UI/UX enhancements, project structure, and local development constraints that were handled during the session.

- **Tool:** Cursor
  Prompt: "in which framework does this navbar-icon-hider-prototype is being built"
  Purpose: To explicitly clarify the frontend framework and build tools utilized by the AI for the scaffolded project.
  Outcome: Useful. The AI directly answered the query, confirming the use of React (TypeScript) and the Vite build tool.

- **Tool:** Cursor
  Prompt: "DOM Path: `div#root > div.layout > main.content > section.panel` make them some more dark so that they will differentiate from others"
  Purpose: To refine the UI's visual hierarchy by modifying the CSS to darken specific DOM elements, improving contrast and visual separation between the sidebar, panel, and page background.
  Outcome: Useful. The AI successfully updated the styling, darkening the background and increasing the border contrast for both the sidebar and the main usage panel as requested.

Bhargav:

Made an Excel clone base for future implementation of different adaptation strategies over it.

AI usage: 100%

- **Entry 1**
  - **Tool:** Gemini 3.1 Pro
  - **Prompt:** "i need some more changes (bug fixes) and advancements for my application. 1) the commands that are in the groups of main tabs (home, insert, draw) shouldn't be interchangeable... 2) space adjustment for groups in ribbon... 3) Duplicate commands: we should not allow same command to be added to a group multiple times... maintain the commands in a class and in a structured way..."
  - **Purpose:** To enforce strict UI rules for default vs. custom tabs, implement dynamic Ribbon resizing (collapsing commands to symbols or overflow boxes), and restructure the codebase for future ML integration.

  - **Outcome:** Partially Used. Logic for non-interchangeable default groups and duplicate command prevention was implemented; initial overflow logic for ribbon shrinking was established.

- **Entry 2**
  - **Tool:** Gemini
  - **Prompt:** "nothing changed at all. you do a check by running yourself and verify... 1) lets project every group as a box... 2) when there is no place for group to hold its commands, entire groups will gracefully collapse into singular dropdown action-buttons... 3) want rename button to perform its functionality... reset button's functionality... remove button should also be able to remove (delete) selected tabs..."
  - **Purpose:** To fix UI bugs where commands were overlapping, implement "Excel-style" group collapsing into floating popovers, and finalize the CRUD (Create, Rename, Update, Delete) functionality for ribbon customization.
  - **Outcome:** Useful. Successfully refined the UI to treat groups as distinct boxes and implemented the popover logic for collapsed groups and the full reset/rename/remove workflow.

- **Entry 3**
  - **Tool:** Gemini
  - **Prompt:** "where is the current configuration of ribbon tabs and groups stored? how can we modify the config with another function or algorithm... 1) find a way to cleanly keep track of total frequency of commands... 2) an effective and appropriate algorithm() which uses that data to compute and create new tab... 3) problem in opening the dropdown groups. they are not showing any box or list..."
  - **Purpose:** To establish a data tracking system for command usage frequency over 3 sessions and design the logic for an "Adaptability Algorithm" that suggests a "Newly Suggested Tab" based on high-frequency features.
  - **Outcome:** Useful. Provided a structured way to log user interactions and the foundational algorithm for the adaptive UI notification system. Fixed the dropdown rendering bug.

- **Entry 4**
  - **Tool:** Gemini
  - **Prompt:** "1) I want it to suggest changes to my workflow at the start of every session... with one more option to whether add this as another new tab or overwrite it... 2) add proper actions and functions to all the functions... no more 'this feature is coming soon' popups. 3) remove that john sudhakar at the top and keep some random name... enable the saving button... 4) modify the default config of home tab to look as it is in the excel's ui..."

- **Purpose:** To polish the user experience by automating suggestions at startup, replacing placeholders with functional Excel-like commands, fixing file-save bugs, and aligning the "Home" tab aesthetics (icons, curves, and labels) with the native MS Excel UI.
- **Outcome:** Useful. Finalized the UI skinning to match Excel closely and ensured the core application logic (saving and command execution) was fully functional.

VII. PROMPTS

Pranav:

- **Tool:** Gemini
  Prompt: "Do everything again. Its not like im averse to using LLMs. I just don't want full thing. Also we want to restrict for Desktop Apps, where we'll have front-end, back-end everything at once. Plus everything should be locally done."
  Purpose: To rewrite a feasibility report and proposal strategy restricting the scope entirely to desktop applications functioning locally.
  Outcome: Partially Used/Modified. The AI generated a plan using Electron.js and local SLMs, but the prompt was iterated upon because it did not target runtime data.

- **Tool:** Gemini
  Prompt: "Nope we want to target runtime, only then we will have data about user right. If it doesn't work we'll anyway change."
  Purpose: To pivot the proposal's strategy strictly to runtime/load-time adaptation so real-time user data could be utilized.
  Outcome: Partially Used/Modified. The user felt the resulting proposal was too specific and committed to high-risk technologies.

- **Tool:** Gemini
  Prompt: "Here is the revised proposal strategy and content. We are pivoting strictly to Runtime/Load-time Adaptation for Desktop Applications (specifically targeting Electron/Node.js apps, as this allows full-stack modification locally). Don't be this specific. Since this is a new topic, there are high chances of failure, and we'd like to not commit to anything before hand"
  Purpose: To generalize the proposal to avoid over-committing to a specific technology stack (like Electron/Node.js) to allow room for failure.
  Outcome: Partially Used/Modified. The user further iterated to soften the commitment to Reinforcement Learning.

- **Tool:** Gemini
  Prompt: "The RL Angle: We stick to 'Reinforcement Learning for Decision Making' because it sounds impressive and fits your 'High Impact' requirement, but we describe the actions broadly (e.g., 'Switching computational fidelity'). Don't commit to this as well. Let's just say as if we are trying to explore"
  Purpose: To frame the Reinforcement Learning aspect strictly as an exploratory research prototype rather than a guaranteed software feature.
  Outcome: Partially Used/Modified. The user later added more details regarding UI/UX adaptations and routing.

- **Tool:** Gemini
  Prompt: "3.1 Problem Statement Current desktop software generally relies on static configuration, assuming a 'standard' execution environment. However, real-world usage involves diverse hardware constraints (varying RAM, CPU, GPU) and user contexts (accessibility needs, network latency). Not just hardware. Since this is only for desktop apps (I am assuming src code will be readily available)? We can just create it properly for changing/modifying frontend and backend code (if possible) and db code (if possible) as well. Let's say one guy uses one feature the most, but it needs a few navs to get there, we can make it easier to get there, and we can mask some features that aren't being used frequently (there has to be a mechanism to reopen removed functionalities as well, as in maybe perform an initial analysis of the src code after installation? And storing it in a structured format)"
  Purpose: To broaden the proposal to include usage-based adaptation, such as re-routing navigation and masking unused features, alongside an initial source code analysis phase.
  Outcome: Partially Used/Modified. The user requested that hardware constraints be re-added and deep research/citations be included.

- **Tool:** Gemini
  Prompt: "Ok now everything is fine, but you can also put some hardware in as well. Also this will come under the research project place. Run a deep research and then give everything (all the Githubs, all the citations etc) as well"
  Purpose: To integrate hardware constraints back into the proposal and generate a comprehensive research plan featuring specific GitHub repositories and citations.
  Outcome: Partially Used/Modified. The user complained about over-committing to specifics again and requested the LaTeX code be completely sanitized.

- **Tool:** Gemini
  Prompt: "Lit Review: Rapid Application Development (RAD)... [Data dump] ...We have chosen MS Excel clone as our prototype for Phase - 1. With all this data, help me create the Release - 1 report. Give me Problem Statement, Proposed Solution, Tech Stack, Team Roles, Milestones & Timeline. Give this in gdocs or any other thing you like"
  Purpose: To synthesize the team's literature review, feasibility study, and status reports into the formal sections required for the Release 1 report.
  Outcome: Useful. The AI generated the content, and the user subsequently pivoted to building a proof of concept.

- **Tool:** Gemini
  Prompt: "Wait. I think before we do this, we need to actually build something. We have the Excel clone (with buttons and tabs), most features don't work but buttons and tabs are there. If possible, according to user's usage, taking in some telemetry, I'd like to check if the app can be dynamically remade, with there obviously being a fallout button... First think, then let's execute, okay? I've attached root directory structure of the Excel clone. Let's use MAPE - K"
  Purpose: To devise a strategic execution plan for a proof-of-concept, adapting an Excel clone's UI dynamically using the MAPE-K architecture and telemetry.
  Outcome: Useful. It initiated a constructive discussion on the architectural flow of the application.

- **Tool:** Gemini
  Prompt: "Sure, let's do all that. But first, do you think you need any files from me? I can provide you. Maybe directory structure, etc as well if you need. First gather everything from me, then let's start implementation"
  Purpose: To inquire what specific files or directory details the AI needed from the user to start the technical implementation.
  Outcome: Useful. The AI provided a list requesting the frontend UI code, Electron main process, and preload scripts.

- **Tool:** Gemini
  Prompt: "Give me a summary of Rainbow paper that I can put under my Lit Review doc"
  Purpose: To obtain a structured summary of the "Rainbow" paper to strengthen the literature review document.
  Outcome: Useful. A detailed summary mapping to the project's architecture was provided.

- **Tool:** Gemini
  Prompt: "Why are we using jscodeshift, electron and all?"
  Purpose: To get solid engineering justifications for the chosen tech stack (jscodeshift, Electron, etc.) for reporting and defense purposes.
  Outcome: Useful. Clear explanations for the technology choices were provided.

Ankit:

- **Tool:** Gemini
  Prompt 1: `read this https://dl.acm.org/doi/epdf/10.1145/3328243.3328252`
  Purpose: Context Engineering. To load the foundational academic paper on Model-Based User Interface Development (MBUID) into the LLM's memory.
  Outcome: Loaded data into context for final summary generation.

- **Tool:** Gemini
  Prompt 2: "how are they going about doing that"
  Purpose: Context Engineering. To force the LLM to extract the specific methodology (Cameleon Reference Framework) from the first paper rather than just the abstract summary.
  Outcome: Loaded data into context.

- **Tool:** Gemini
  Prompt 3: "did they actually do this or was this a feasibility study"
  Purpose: Context Engineering & Validation. To determine if the referenced paper provided a production-ready framework or just theoretical groundwork.
  Outcome: Loaded data into context. Clarified that the approach was a feasibility study, preventing the LLM from hallucinating production-ready tools later on.

- **Tool:** Gemini
  Prompt 4: "is there a implementation you can find anywhere? even small scaled"
  Purpose: Context Engineering. To ground the theoretical context with actual, functional tools (like USIXML or Eclipse EMF) that utilize these models.
  Outcome: Useless, irrelevant.

- **Tool:** Gemini
  Prompt 5: "How about this [Attached File: Situation_Analytics_and_Model-based_User_Interface.pd
  Purpose: Context Engineering. To expand the LLM's knowledge base from static device adaptation to dynamic, runtime adaptation based on user situations (SitAdapt 2.0).
  Outcome: Loaded data into context.

- **Tool:** Gemini
  Prompt 6: "is there something that would adapt software based on usage patterns? Something like dropping or hiding features (to simplify UI) or just dropping sections of code (to make more efficient) or adapt UI to make it more productive for the user"
  Purpose: Project Pivot. To steer the LLM away from general UI models and specifically toward the core idea: usage-based feature hiding and code dropping (debloating).
  Outcome: Useful. The LLM successfully transitioned to discussing Self-Adaptive Systems and UI simplification.

- **Tool:** Gemini
  Prompt 7: "give me papers for these"
  Purpose: Context Engineering. To gather specific academic literature validating the core idea of usage-based adaptation and code pruning.
  Outcome: Useful. Generated a list of highly relevant papers divided into UI simplification and code debloating.

- **Tool:** Gemini
  Prompt 8: "Go through every single paper you mentioned in your previous response about 1. UI simplification and give me a long summary"
  Purpose: Context Engineering. To deeply load the methodologies of the UI simplification papers (Feature Models, Levenshtein method) into the active context.
  Outcome: Useful. Provided detailed extraction of how UI adaptation works in practice.

- **Tool:** Gemini
  Prompt 9: "For your code pruning, nothing is about what I asked. Others are about neural networks, I wanted about Code/feature pruning on a desktop app or website or something based on how users use the features"
  Purpose: Prompt Iteration / Course Correction. The LLM hallucinated/assumed AI model pruning. This prompt corrected the trajectory specifically back to desktop application debloating based on user clicks.
  Outcome: Modified. The LLM corrected itself and introduced the accurate concepts of "Software Debloating" (e.g., JRED).

- **Tool:** Gemini
  Prompt 10: "go through all these and mention: 1. summary 2. how are they doing 3. did they actually do this or is it just a study in detail [Attached 4 PDFs: menu adaptation, web usage mining, JRED, debloat 2024]"
  Purpose: Context Engineering. To load the final, highly specific papers on gradual UI fading, usage mining, Java debloating, and debloating tool evaluations.
  Outcome: Useful. Fully primed the LLM with the exact constraints, successes, and

failures of the proposed technical space.

- **Tool:** Gemini
  Prompt 11: "Make a google doc based on this. A feasibility study on my idea. Mention this research summaries for both UI simplification and this recent 4 paper summaries necessarily"
  Purpose: Final Artefact Generation. To synthesize all the previously loaded context (10 steps of engineering) into a cohesive feasibility study.
  Outcome: Useful. The LLM generated a comprehensive feasibility study that combined both tracks (UI and Code).

- **Tool:** Gemini
  Prompt 12: "we are making an adaptive software on offline desktop apps we will have a: 1. Demo app (Assuming on the desktop we will have the compiling tools and source code) 2. Statistics and tracking module... 3. Code read and write module... 4. Code editor... find more assumptions other than assuming having the sourcecode and the full toolchain like java"
  Purpose: Technical Brainstorming. To test the proposed architecture against the academic literature loaded earlier and identify hidden technical prerequisites.
  Outcome: Useful. Identified critical flaws like native code blind spots and compilation overhead.

- **Tool:** Gemini
  Prompt 13: "OH, then you say this is infeasible"
  Purpose: Stress Testing. To challenge the LLM's assessment and force it to explicitly define the line between "complex" and "impossible."
  Outcome: Useless, still said it is possible when it was not.

- **Tool:** Gemini
  Prompt 14: "but then us needing the whole java toolchain installed in there PLUS our own editing software constructing ASTs and stuff for probably 40k to 100k lines of code is just too much RAM and compute and the goal of this was to make heavier software run better on resource constrained laptops"
  Purpose: Constraint Injection. The LLM did not consider the benefits and costs.
  Outcome: Useful. The LLM pivoted away from heavy AST compilation toward lightweight class-loading and binary rewriting.

- **Tool:** Gemini
  Prompt 15: "so focusing on UI fixes most stuff? and for UI too we need to process on remote servers?"
  Purpose: Architecture Refinement. To confirm if the UI-only route solves the resource problem and to establish offline capabilities.
  Outcome: Useful. Confirmed a "No-Compile" offline UI adaptation route using configuration files.

- **Tool:** Gemini
  Prompt 16: "new plan: UI Adapter Modules: 1. Demo app... 2. Statistics and tracking module... 3. Flipper module (basically a config file)... 4. Coding guidelines document... Should we go this route?"

Purpose: Architecture Validation. To propose a finalized, lightweight, developer-guided architecture and seek validation based on the academic context.
Outcome: Useful. The plan was validated as the optimal, stable approach (aligning with Software Product Line engineering).

- **Tool:** Gemini
  Prompt 17: "what will each module do in detail? and we still assume that sourcecode exists right?"
  Purpose: Specification Generation. To break down the validated architecture into actionable development specs for each module.
  Outcome: Useful. Generated detailed roles and operational mechanics for the 4 modules.

- **Tool:** Gemini
  Prompt 18: "give me the full workflow something like: App is running... and now so what was the use of flagging sections of code? and can we really just have a switch that moves a menu/button item from anywhere into a hidden tab?"
  Purpose: Workflow Mapping & Feasibility Check. To map the user data flow from end-to-end and clarify the technical feasibility of dynamic UI layouts.
  Outcome: Useful. Confirmed the necessity of the Command Pattern and dynamic grids for moving UI elements.

- **Tool:** Gemini
  Prompt 19: "what demo app would be ideal to test this on? (we would make that app)"
  Purpose: MVP Definition. To identify the best type of application to build that would explicitly showcase the framework's ability to save RAM and simplify UIs.
  Outcome: Useful. Resulted in the "OmniTool Desktop" concept (Document + Image + System util).

- **Tool:** Gemini
  Prompt 20: "give a prompt to give to antigravity so it makes the app. let it have all those features flagged and a config file that has all those flags : bool"
  Purpose: Prompt Generation (Meta-Prompting). To use the LLM to write a highly technical prompt for another AI coding agent to actually build the demo app, encompassing all the architectural rules established in the session.
  Outcome: Useful. The final deliverable was generated.

Karthik:

Did not use AI here. Only Cursor for development, declared in the previous section.

Nilanjan:

- **Entry 1**
  - **Tool:** Gemini
  - **Prompt:** "I added a new tab element to the defaultRibbonConfig, but it is not getting rendered as a separate tab. Why?"
  - **Purpose:** To troubleshoot why a newly added UI tab in a React configuration file wasn't displaying correctly.

  - **Outcome:** Partially used. The output correctly identified a duplicate Tab ID issue, which was fixed, but it required further iteration as the component was still not rendering properly.

- **Entry 2**
  - **Tool:** Gemini
  - **Prompt:** (Updated code with a unique tab ID `id: 'Hello'`) "Still, it is not getting rendered."
  - **Purpose:** To follow up on the previous rendering issue after fixing the tab ID collision, attempting to find the root cause of the broken UI.
  - **Outcome:** Useful. The output identified that duplicate Group IDs were overwriting React DOM references, and the provided config with strictly unique IDs resolved the rendering issue.

- **Entry 3**
  - **Tool:** Gemini
  - **Prompt:** "How are the buttons getting rendered in this file? (Referring to Ribbon.tsx)"
  - **Purpose:** To understand the specific data-driven rendering logic, state management, and the `renderCommand` factory function within the `Ribbon.tsx` component.
  - **Outcome:** Useful. Provided a clear breakdown of how the component maps data to visuals, handles states, and manages popovers.

- **Entry 4**
  - **Tool:** Gemini
  - **Prompt:** "Add a field called clicks in RibbonCommandDefinition and initialize it to zero for every command in the initializeCommands function."
  - **Purpose:** To update the data model and state initialization logic to support click-tracking for user analytics.
  - **Outcome:** Useful. Provided an efficient, programmatic solution using `Object.keys().reduce` to inject the new field without requiring manual updates to 50+ lines of code.

- **Entry 5**
  - **Tool:** Gemini
  - **Prompt:** "I want to use these clicks to render a new 'Frequent' tab showing the top 5 most clicked buttons, and I want them all to render as Large buttons."
  - **Purpose:** To build a dynamic UI feature (a "Frequent" tab) that sorts data by the new click metric and overrides default button sizing.

  - **Outcome:** Useful. Provided the sorting/filtering logic using `useMemo` and the necessary UI overrides (`forceLarge`) for the render function.

- **Entry 6**
  - **Tool:** Gemini
  - **Prompt:** "How can I persist these clicks in an Electron.js app without using localStorage? Can I store them in a file?"
  - **Purpose:** To implement a persistent, file-based data storage solution for the analytics data within an Electron environment.
  - **Outcome:** Useful. Provided file system (`fs`) logic to save data in the AppData folder, refactored the registry to load/save, and included relevant security best practices for Electron.

Bhargav:

(No additional prompt entries beyond those listed in the AI Usage Declaration section.)

VIII. MILESTONES & TIMELINE

The following timeline outlines our project lifecycle, transitioning from our initial UI-focused prototypes (Release 1) to a generalized, backend-optimized adaptive framework (Release 2 and Final).

Phase	Milestone & Specific Deliverables	Target Date	Assigned Members
Release 1	**Architecture & UI Prototypes:** Delivery of this proposal, validation of the MAPE-K architecture, and completion of three distinct UI adaptation prototypes (Spreadsheet Testbed).	Mar 17, 2026	All Members
Sprint 1	**Backend Infrastructure:** Implement the core configuration file parser and establish the feature-flag toggling API bridging Node.js and the frontend.	Apr 01, 2026	Pranav, Ankit, Nilanjan
Release 2	**Framework Abstraction & Backend Debloating:** MAPE-K engine abstracted into a reusable library. Demonstration of active backend memory purging. Integration with a new, secondary testbed.	Apr 15, 2026	All Members
Sprint 2	**Performance Evaluation:** Conduct system profiling (RAM/CPU metrics) and user testing (Task Completion Time) across both static and adaptive states.	Apr 29, 2026	Karthik, Bhargav, Ankit
Final	**Code Freeze & Final Polish:** Final bug fixes, comprehensive documentation, and production of the project demonstration video.	May 10, 2026	All Members

Table 1: Project Milestones & Timeline

IX. GOALS FOR RELEASE - 2

- The full adaptable software development guidelines document.
- Development of the complete prototype of adaptable software to prove these guidelines work:
  - A desktop application (main prototype) — Excel clone. Most features on Excel are unused. This is the ideal candidate for adaptability.
  - A minimal mobile application as a POC for showing feasibility.
  - A minimal web application as a POC for showing feasibility.